


SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)




THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

# END-SEMESTER PROJECT REPORT

on

# MAT340 – REINFORCEMENT LEARNING

**Submitted By**

| | | |
|---|---|---|
| **Student Name** | : | Sanjay Shrinivas |
| **Register Number** | : | 127177025 |
| **Programme** | : | M.Sc. Data Science (5 Years Integrated) |
| **Academic Year** | : | 2026 – 2027 |
| **School** | : | School of Arts, Sciences, Humanities & Education |
| **Project Title** | : | A Comparative Study of Q-Learning and SARSA for Energy Consumption Management Using Tabular Reinforcement Learning |

**External Faculty 1 Signature** **External Faculty 2 Signature**

# Contents

**Abstract** ..... **6**

**Chapter 1 Introduction** ..... **7**

1.1 Background ..... 7

1.2 Reinforcement Learning ..... 7

1.3 Temporal-Difference Control ..... 7

1.4 Reinforcement Learning for Energy Management ..... 8

1.5 Project Overview ..... 8

**Chapter 2 Problem Statement** ..... **10**

**Chapter 3 Objectives** ..... **11**

**Chapter 4 Dataset Description** ..... **12**

4.1 Overview of the Dataset ..... 12

4.2 Data Validation and Cleaning ..... 12

4.3 Exploratory Data Analysis ..... 13

4.4 Chronological Training and Test Split ..... 14

4.5 State Discretisation ..... 14

4.6 Role of the Dataset in the Project ..... 15

**Chapter 5 Methodology** ..... **16**

5.1 System Architecture ..... 16

5.2 System Workflow ..... 16

5.3 Data Preprocessing ..... 17

5.4 Replay Environment ..... 17

5.5 Action Space ..... 18

5.6 Reward Function ..... 18

5.7 Q-Learning ..... 18

5.8 SARSA ..... 18

5.9 Exploration Strategy ..... 19

5.10 Model Training ..... 19

5.11 Model Evaluation ..... 19

**Chapter 6 Experimental Setup** ..... **20**

6.1 Training Configuration ..... 20

6.2 Training Procedure ..... 20

6.3 Fairness and Reproducibility Controls ..... 20

6.4 Software and Hardware Configuration ..... 21

**Chapter 7 Evaluation Metrics** ..... **22**

7.1 Episode Reward ..... 22

7.2 Mean and Standard Deviation of Test Reward ..... 22

7.3 Moving-Average Training Reward ..... 22

7.4 Policy Agreement ..... 22
7.5 Action Distribution ..... 22
7.6 Robustness Across Seeds ..... 22
7.7 Computational Cost ..... 22
**Chapter 8 Results and Analysis ..... 23**
8.1 Learned Q-Tables ..... 23
8.2 Learned Policies ..... 24
8.3 Training Behaviour ..... 24
8.4 Test Performance in the Primary Run ..... 25
8.5 Five-Seed Robustness Evaluation ..... 25
8.6 Final Comparison ..... 27
**Chapter 9 Sample Policy Outputs ..... 28**
9.1 Recommended Action per Consumption State ..... 28
9.2 Interpretation of the Sample Outputs ..... 28
**Chapter 10 Discussion ..... 29**
10.1 Comparison of Q-Learning and SARSA ..... 29
10.2 Why the Two Algorithms Behave So Similarly ..... 29
10.3 Sanity Check of the Learned Policy ..... 29
10.4 Interpretation of the Seed Variability ..... 30
10.5 Computational Considerations ..... 30
10.6 Final Interpretation ..... 30
**Chapter 11 Limitations ..... 31**
**Chapter 12 Future Enhancements ..... 32**
**Chapter 13 Conclusion ..... 33**
**References ..... 34**
**Appendix A Source Code ..... 35**

# List of Figures

Figure 4.1: Distribution of Global Active Power ..... 13
Figure 4.2: Household Global Active Power Over Time ..... 14
Figure 4.3: Energy State Counts, Training vs Test ..... 15
Figure 5.1: Workflow of the Energy Management RL Study ..... 17
Figure 8.1: Q-Table Heatmaps of Q-Learning and SARSA (seed 42) ..... 23
Figure 8.2: Training Reward (25-Episode Moving Average) ..... 24
Figure 8.3: Test Reward Across Five Seeds (Mean ± SD) ..... 26
Figure 8.4: Final Q-Learning vs SARSA Comparison (Mean ± SD over Five Seeds) ..... 27

# List of Tables

Table 4.1: Original Dataset Information ..... 12
Table 4.2: Dataset Size Before and After Cleaning ..... 12
Table 4.3: Descriptive Statistics of the Cleaned Dataset (2,049,280 records each) ..... 13
Table 4.4: Chronological Training and Test Split ..... 14
Table 4.5: State Distribution in the Training and Test Subsets ..... 15
Table 5.1: Action Space ..... 18
Table 5.2: Reward Rules ..... 18
Table 6.1: Experimental Configuration ..... 20
Table 8.1: Q-Learning Q-Table (seed 42) ..... 23
Table 8.2: SARSA Q-Table (seed 42) ..... 23
Table 8.3: Greedy Policies Learned in the Primary Run ..... 24
Table 8.4: Primary-Run Results (seed 42, 100 test windows) ..... 25
Table 8.5: Action Counts During Test Evaluation (100 windows x 200 steps) ..... 25
Table 8.6: Results for Five Independent Seeds ..... 25
Table 8.7: Summary Over Five Seeds ..... 26
Table 8.8: Final Comparison of Q-Learning and SARSA ..... 27
Table 9.1: Policy Learned in the Primary Run ..... 28

# Abstract

Reinforcement Learning (RL) is a branch of machine learning in which an agent learns a decision-making policy by interacting with an environment and receiving numerical rewards. This case study presents a comparative evaluation of two classical tabular temporal-difference (TD) control algorithms, Q-Learning (off-policy) and SARSA (on-policy), applied to an energy consumption management task built from real household electricity measurements. The study compares their learning behaviour, learned policies, test performance, robustness across random seeds and computational cost under a common, reproducible experimental setting.

The dataset is the Household Electric Power Consumption data set, containing 2,075,259 minute-level records. After removing 25,979 records with missing or invalid values, 2,049,280 records are retained, sorted chronologically and divided into the earliest 80% (1,639,424 records) for training and the latest 20% (409,856 records) for testing. The global active power signal is discretised into five consumption states (Very Low, Low, Medium, High, Very High) using quantile thresholds of 0.272, 0.400, 1.266 and 1.742 kW learned from the training data only. The agent chooses among three actions (Reduce, Maintain, Allow Higher Consumption) in a historical-data replay environment with a transparent, manually designed reward function.

Both algorithms are trained for 500 episodes of 200 steps with identical hyperparameters ($\alpha$ = 0.10, $\gamma$ = 0.95, $\varepsilon$ decaying from 1.0 with a factor of 0.995). In the primary run (seed 42), both learn the same greedy policy (Maintain in the Very Low, Low and Very High states; Reduce in the Medium and High states) and obtain an identical mean test reward of 288.92 per 200-step window. Training takes 3.897 s for Q-Learning and 3.487 s for SARSA. Across five independent seeds, the mean test rewards are 319.33 (SD 61.93) for Q-Learning and 323.97 (SD 65.11) for SARSA, a difference of only 4.64 reward units that is far smaller than the seed-to-seed variability. The two algorithms therefore cannot be separated statistically in this setting.

The study concludes that, in a replay environment where actions do not influence future states, Q-Learning and SARSA behave almost identically, and the observed differences come mainly from random exploration rather than from the algorithms themselves. All rewards are arbitrary scores and do not represent real electricity savings.

**Keywords:** Reinforcement Learning, Q-Learning, SARSA, Temporal-Difference Learning, Energy Management, Household Power Consumption, $\varepsilon$-Greedy Exploration.

# Chapter 1 Introduction

## 1.1 Background

Electricity demand in households varies continuously with occupant behaviour, appliance usage, season and time of day. Managing this demand is important for reducing cost, flattening peak load and lowering carbon emissions. Smart meters now record household consumption at fine time resolution, producing large time series that can be used to study demand patterns and to design automated energy-management strategies.

Traditional approaches to demand management rely on fixed schedules, rule-based controllers or supervised forecasting models. Rule-based controllers are simple but cannot adapt to changing behaviour, while forecasting models predict consumption but do not by themselves decide what action to take. Reinforcement learning addresses the decision-making aspect directly: an agent observes the current situation, selects an action, receives a reward and gradually learns which actions give the best long-term outcome.

## 1.2 Reinforcement Learning

Reinforcement learning is formulated as a Markov Decision Process (MDP) defined by a set of states S, a set of actions A, a transition function P(s' | s, a), a reward function R(s, a, s') and a discount factor $\gamma \in [0, 1)$. At each time step t the agent observes a state $s_t$, chooses an action $a_t$, receives a reward $r_{t+1}$ and moves to a new state $s_{t+1}$. The objective is to learn a policy $\pi(a \mid s)$ that maximises the expected discounted return

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{1.1}$$

The action-value function $Q_\pi(s, a)$ is the expected return obtained by taking action a in state s and following the policy $\pi$ afterwards. If Q is known, an optimal policy simply selects the action with the largest Q-value in every state.

A central difficulty in RL is the exploration–exploitation trade-off: the agent must try unfamiliar actions to discover better ones, yet also use what it has learned to collect reward. The $\varepsilon$-greedy rule used in this study chooses a random action with probability $\varepsilon$ and the greedy action otherwise, with $\varepsilon$ reduced gradually during training.

## 1.3 Temporal-Difference Control

Temporal-difference (TD) methods learn directly from experience without a model of the environment. They update an estimate toward a target built from the observed reward and the current estimate of the next state, a process called bootstrapping. Two of the best-known TD control algorithms are Q-Learning and SARSA.

**Q-Learning** is an off-policy algorithm. It updates the value of the action taken toward the best possible next action, regardless of what the behaviour policy actually does next:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[ r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{1.2}$$

**SARSA** (State–Action–Reward–State–Action) is an on-policy algorithm. It updates toward the value of the next action that is actually selected by the exploring policy:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[ r + \gamma Q(s',a') - Q(s,a) \right] \tag{1.3}$$

Because SARSA accounts for exploratory moves, it tends to learn more conservative policies in risky environments (the classic cliff-walking example), whereas Q-Learning learns the value of the greedy policy and can be more aggressive. How much this theoretical difference matters in practice depends strongly on the environment, which is the main question examined here.

## 1.4 Reinforcement Learning for Energy Management

Energy management is a natural application for RL because it is a sequential decision problem with delayed consequences: reducing consumption during a high-demand period may cost comfort now but save money or reduce grid stress later. In this study the household consumption signal is converted to five discrete categories, and the agent decides whether to reduce, maintain or allow higher consumption in each category.

An important feature of this case study is that it uses a **historical replay environment**. The next state is simply the next recorded consumption category in the time series, regardless of the action chosen. The reward is a manually designed score. The experiment therefore evaluates the learning algorithms on recorded data; it does not estimate real electricity savings because the actions do not alter future consumption.

## 1.5 Project Overview

The project implements tabular Q-Learning and SARSA from first principles in Python with NumPy and compares them on a common task. The complete pipeline is:

1. Load the household_power_consumption.txt dataset.
2. Validate, clean and chronologically sort the records.
3. Perform exploratory data analysis of the consumption signal.
4. Split the data chronologically into 80% training and 20% test data.
5. Discretise global active power into five states using training-only quantile thresholds.
6. Build a replay environment with three actions and an explicit reward function.
7. Train Q-Learning and SARSA with identical settings and seeded random generators.
8. Plot training reward curves and inspect the learned Q-tables.
9. Evaluate the greedy policies on 100 fixed, non-overlapping chronological test windows.

10. Repeat training and evaluation for five independent seeds to assess robustness.
11. Export all tables and figures for reporting.

In the primary experiment both algorithms learn the same policy and obtain a mean test reward of 288.92 per window. Over five seeds the mean test rewards are 319.33 (Q-Learning) and 323.97 (SARSA), with standard deviations above 60, so the difference is not meaningful.

# Chapter 2 Problem Statement

The objective of this case study is to design and evaluate a reinforcement-learning agent that recommends an energy-consumption action based on the current household consumption level, and to compare two TD control algorithms, Q-Learning and SARSA, on the same task.

Given the current consumption state, the agent must select one of the following actions:

1. Reduce Consumption
2. Maintain Consumption
3. Allow Higher Consumption

The decision problem can be written as

$$\pi(s) \to a, \quad s \in \{0, 1, 2, 3, 4\}, \quad a \in \{0, 1, 2\} \tag{2.1}$$

where s is the discretised consumption state (0 = Very Low, 1 = Low, 2 = Medium, 3 = High, 4 = Very High), a is the chosen action and π is the learned policy. The agent must learn from the training portion of the data and apply its policy to a later, unseen period.

The effectiveness of each algorithm is assessed using the mean and variability of episode reward on held-out test windows, the shape of the training reward curve, the learned Q-tables and policies, agreement between the two algorithms, robustness over multiple random seeds and training time. The central question is whether the on-policy and off-policy update rules lead to measurably different outcomes on this task.

# Chapter 3 Objectives

The major objectives of the project are:

1. To understand the principles of reinforcement learning, Markov Decision Processes and temporal-difference control.
2. To study the working of Q-Learning (off-policy) and SARSA (on-policy) and the differences between their update rules.
3. To prepare a real household electricity-consumption dataset of 2,075,259 minute-level records for reinforcement-learning experiments.
4. To split the data chronologically and derive state thresholds from training data only, avoiding information leakage.
5. To design a transparent replay environment with five states, three actions and an explicit rule-based reward function.
6. To implement tabular Q-Learning and SARSA with ε-greedy exploration, decaying ε and random tie-breaking.
7. To train both algorithms under identical hyperparameters and seeded random generators for a fair comparison.
8. To evaluate the greedy policies on identical, non-overlapping chronological test windows.
9. To assess robustness by repeating the experiment with five independent random seeds.
10. To analyse the learned Q-tables and policies and to interpret the results honestly, including what the experiment cannot show.
11. To document limitations and propose realistic directions for future work.

# Chapter 4 Dataset Description

## 4.1 Overview of the Dataset

The dataset used in this case study is the Household Electric Power Consumption data set, supplied as the text file household_power_consumption.txt. It contains minute-level measurements of electric power consumption in a single household, recorded from December 2006 (the first record is 16/12/2006 17:24:00) until late 2010. The file is semicolon-separated and uses "?" to mark missing values. It has 2,075,259 records and nine columns.

Table 4.1: Original Dataset Information

| Attribute | Description |
|---|---|
| File name | household_power_consumption.txt |
| File size | about 133 MB |
| Sampling interval | 1 minute |
| Number of records | 2,075,259 |
| Number of columns | 9 |
| Missing-value marker | ? |
| Variable used for states | Global_active_power (kW) |

The nine columns are Date, Time, Global_active_power (kW), Global_reactive_power (kVAR), Voltage (V), Global_intensity (A), and Sub_metering_1, Sub_metering_2 and Sub_metering_3 (watt-hours of active energy, covering the kitchen, laundry room, and water heater/air conditioner respectively). The state variable used by the agent is Global_active_power, the household's total active power draw.

## 4.2 Data Validation and Cleaning

The cleaning procedure first checks that all required columns are present. The Date and Time columns are combined and parsed into a single Datetime column (day-first format), and all measurement columns are converted to numeric values, with invalid entries turned into missing values. Records with a missing timestamp or measurement are dropped, the remaining records are sorted chronologically, duplicate timestamps are removed and the index is reset. A safeguard raises an error if fewer than 1,000 valid rows remain.

Table 4.2: Dataset Size Before and After Cleaning

| Description | Number of Records |
|---|---|
| Original dataset | 2,075,259 |
| Removed (missing, invalid or duplicate) | 25,979 |
| Cleaned dataset | 2,049,280 |
| Missing values remaining | 0 |
| Duplicate timestamps remaining | 0 |

About 1.25% of the records were removed. Sorting before splitting is essential because the experiment relies on chronological order.

## 4.3 Exploratory Data Analysis

Summary statistics of the cleaned numeric variables are given in Table 4.3.

Table 4.3: Descriptive Statistics of the Cleaned Dataset (2,049,280 records each)

| Variable | Mean | Std | Min | 25% | 50% | 75% | Max |
|---|---|---|---|---|---|---|---|
| Global_active_power | 1.092 | 1.057 | 0.076 | 0.308 | 0.602 | 1.528 | 11.122 |
| Global_reactive_power | 0.124 | 0.113 | 0.000 | 0.048 | 0.100 | 0.194 | 1.390 |
| Voltage | 240.840 | 3.240 | 223.200 | 238.990 | 241.010 | 242.890 | 254.150 |
| Global_intensity | 4.628 | 4.444 | 0.200 | 1.400 | 2.600 | 6.400 | 48.400 |
| Sub_metering_1 | 1.122 | 6.153 | 0.000 | 0.000 | 0.000 | 0.000 | 88.000 |
| Sub_metering_2 | 1.299 | 5.822 | 0.000 | 0.000 | 0.000 | 1.000 | 80.000 |
| Sub_metering_3 | 6.458 | 8.437 | 0.000 | 0.000 | 1.000 | 17.000 | 31.000 |

The global active power has a mean of 1.092 kW and a median of 0.602 kW. Because the mean is well above the median and the maximum (11.122 kW) is about ten times the mean, the distribution is strongly right-skewed. The histogram of global active power (Figure 4.1) shows a large concentration of low-power readings (baseline load) with a second, smaller peak around 1.5 kW and a long tail of rare high-power events. The time-series plot (Figure 4.2) shows persistent short-term fluctuation with clear seasonal variation over the four years, including a pronounced dip in mid-2008, which is consistent with a period of reduced household usage.

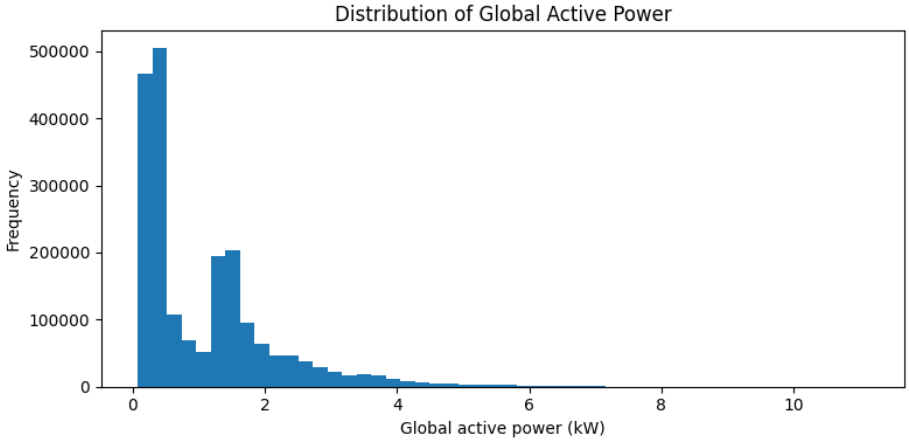



Figure 4.1: Distribution of Global Active Power

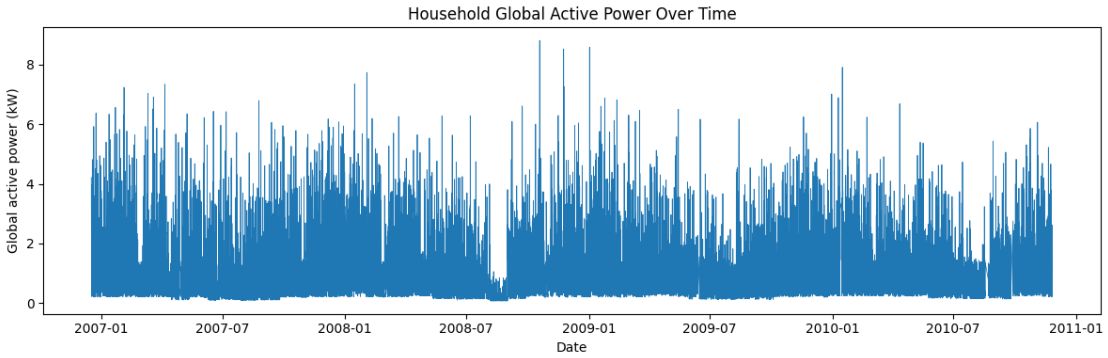



Figure 4.2: Household Global Active Power Over Time

This skewed, multi-modal distribution is the reason quantile-based discretisation is used for the states: equal-width bins would put most observations in the lowest bins and almost none in the highest ones.

## 4.4 Chronological Training and Test Split

Because the data form a time series, a random split would let the model see the future during training. The data are therefore divided chronologically: the earliest 80% of records are used for training and the latest 20% are held out for testing.

Table 4.4: Chronological Training and Test Split

| Subset | Records | Share |
|---|---|---|
| Training | 1,639,424 | 80.0% |
| Test | 409,856 | 20.0% |
| Total | 2,049,280 | 100.0% |

## 4.5 State Discretisation

Tabular RL requires a finite state space. Global active power is converted to five states using the 20th, 40th, 60th and 80th percentiles of the **training** power values only, so that no test information enters the thresholds. The resulting thresholds are 0.272, 0.400, 1.266 and 1.742 kW, and states are assigned with np.digitize:

$$s = \begin{cases} 0 & \text{if } p < 0.272 \\ 1 & \text{if } 0.272 \le p < 0.400 \\ 2 & \text{if } 0.400 \le p < 1.266 \\ 3 & \text{if } 1.266 \le p < 1.742 \\ 4 & \text{if } p \ge 1.742 \end{cases} \tag{4.1}$$

Table 4.5: State Distribution in the Training and Test Subsets

| State | ID | Training count | Test count |
|---|---|---|---|
| Very Low | 0 | 324,442 | 63,982 |
| Low | 1 | 329,826 | 84,840 |
| Medium | 2 | 328,601 | 106,047 |
| High | 3 | 328,271 | 89,978 |
| Very High | 4 | 328,284 | 65,009 |

The training set is almost perfectly balanced across the five states (about 20% each), as expected from quantile binning. The test set is less balanced: Medium is the most frequent state (106,047 records, about 25.9%) and Very Low the least frequent (63,982 records, about 15.6%). This shift shows that consumption patterns in the later period differ somewhat from those in the earlier period, which makes the chronological evaluation a realistic test of generalisation.

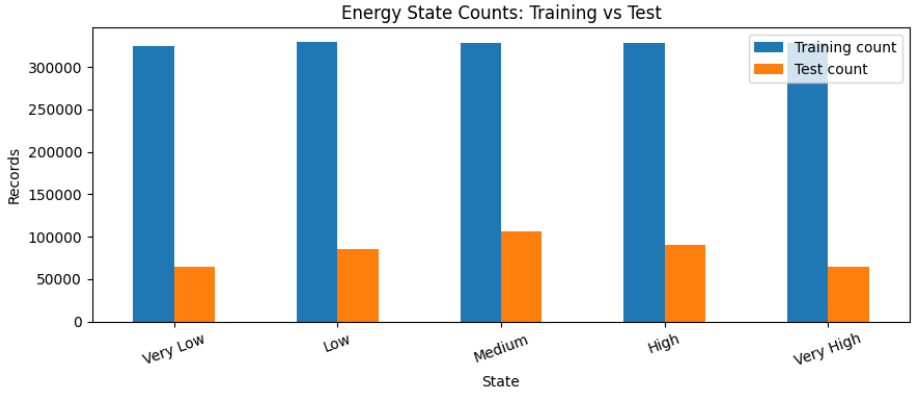



Figure 4.3: Energy State Counts, Training vs Test

## 4.6 Role of the Dataset in the Project

The dataset provides a real, long and noisy consumption sequence from which the state sequence is derived. It supplies both the training experience and the held-out test sequence for the replay environment. It does not provide any information on how consumption would respond to an action, which is the most important limitation of the study and is discussed in Chapter 11.

# Chapter 5 Methodology

## 5.1 System Architecture

The proposed system consists of the following components:

1. **Data Loading:** read the semicolon-separated power-consumption file.
2. **Validation and Cleaning:** parse timestamps, convert numeric columns, remove invalid rows and duplicates, sort chronologically.
3. **Chronological Split:** earliest 80% for training, latest 20% for testing.
4. **State Discretisation:** five quantile-based states from training data only.
5. **Replay Environment:** a class that steps through the recorded state sequence and returns the next state, reward and a done flag.
6. **Learning Agents:** tabular Q-Learning and SARSA with a 5 x 3 Q-table.
7. **Exploration:** ε-greedy action selection with decaying ε and random tie-breaking.
8. **Evaluation:** deterministic greedy evaluation on fixed non-overlapping test windows.
9. **Robustness Analysis:** repetition over five independent seeds.
10. **Reporting:** tables, plots and CSV exports.

## 5.2 System Workflow

The workflow starts with loading and cleaning the data, followed by the chronological split and discretisation. The training state sequence is wrapped in a replay environment. For each of 500 training episodes, a random starting point is drawn and the agent interacts with the environment for 200 steps, updating its Q-table at every step. After training, the greedy policy of each algorithm is applied to the same 100 test windows and the episode rewards are compared. The whole process is repeated for five seeds.

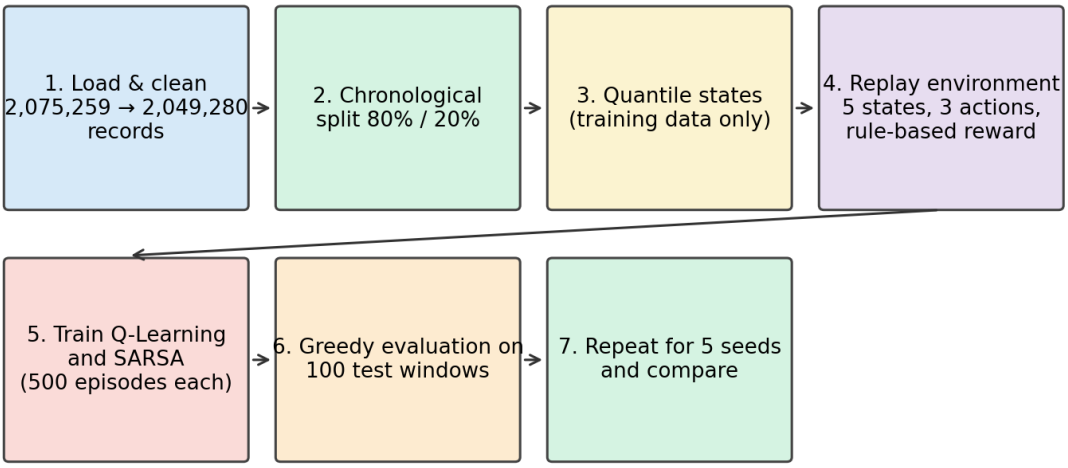



Figure 5.1: Workflow of the Energy Management RL Study

## 5.3 Data Preprocessing

Preprocessing consists of timestamp parsing (day-first), numeric conversion with coercion of invalid entries, removal of rows with missing values, chronological sorting and removal of duplicate timestamps. Only the Global_active_power column is later used to construct states; the other variables are retained for exploratory analysis. No scaling is needed because the state is categorical.

## 5.4 Replay Environment

The environment is implemented as the class EnergyReplayEnvironment. It stores a one-dimensional array of state identifiers (0–4) and a pointer to the current position. The key design choices are:

- **State:** the consumption category at the current time step.
- **Transition:** after any action, the pointer moves one step forward and the next state is the next recorded category. The action has no influence on it.
- **Episode:** a window of up to 200 consecutive steps. At the start of each training episode a start index is drawn uniformly at random from all positions that leave room for a full episode.
- **Termination:** an episode ends when 200 steps are completed or the end of the sequence is reached.
- **Validation:** the class raises errors for state identifiers outside 0–4, invalid action numbers, non-positive episode length and too-short sequences.

Training uses the training state sequence (1,639,424 steps); evaluation creates a fresh environment over the test sequence for each window.

## 5.5 Action Space

The agent chooses among three actions:

Table 5.1: Action Space

| ID | Action | Meaning |
|---|---|---|
| 0 | Reduce Consumption | Recommend lowering demand |
| 1 | Maintain Consumption | Recommend keeping demand unchanged |
| 2 | Allow Higher Consumption | Permit demand to rise |

## 5.6 Reward Function

The reward is a transparent rule-based score computed from the current state s, the next observed state s' and the action a. The units are arbitrary and are not kWh, currency or measured savings.

Table 5.2: Reward Rules

| Action | Base score | Transition adjustment |
|---|---|---|
| Reduce (0) | +2 if s >= 2 (Medium or above), else 0 | +1 if s' < s; −1 if s' > s; 0 if unchanged |
| Maintain (1) | +1 if s <= 2 (Medium or below), else −1 | +0.5 if the state changes by at most one level |
| Allow Higher (2) | −2 if s >= 2, else 0 | −1 if s' > s; otherwise 0 |

The intent is that reducing consumption is rewarded when demand is already medium or high, maintaining is rewarded when demand is low or medium, and allowing higher consumption is penalised when demand is already high. Because the reward is designed by hand, any conclusions are conclusions about the reward design, not about real energy savings.

## 5.7 Q-Learning

Q-Learning maintains a 5 x 3 table of action values initialised to zero. At each step the agent chooses an action with an ε-greedy rule, observes the reward r and next state s', and updates

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[ r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{5.1}$$

At the final step of an episode the target is simply r. The maximum over the next-state values makes the update independent of the next action actually taken, so Q-Learning is off-policy.

## 5.8 SARSA

SARSA uses the same table and exploration rule, but selects the next action a' before the update and uses its value:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)] \tag{5.2}$$

The next action a' becomes the action executed at the following step, so the update follows the behaviour policy exactly and SARSA is on-policy. At the terminal step the target is r.

## 5.9 Exploration Strategy

Both algorithms use ε-greedy action selection. With probability ε a random action is chosen; otherwise the action with the largest Q-value is chosen, with ties broken at random (using np.isclose) to avoid a systematic bias toward the first action when values are equal. ε starts at 1.0, is multiplied by 0.995 after every episode and is never allowed to fall below 0.01.

After 500 episodes ε has decayed to about $0.995^{500} = 0.082$, so it never reaches its 0.01 floor and the agent is still exploring roughly 8% of the time at the end of training. This matters for interpreting the training curves and for the on-policy behaviour of SARSA.

## 5.10 Model Training

Both algorithms are trained for 500 episodes of 200 steps, giving 100,000 value updates each, with learning rate $\alpha = 0.10$ and discount factor $\gamma = 0.95$. A single random generator, created with np.random.default_rng(seed), controls both the choice of episode start points and all exploratory and tie-breaking decisions. The primary experiment uses seed 42 and the robustness study uses seeds 0–4. Training time is measured with time.perf_counter().

## 5.11 Model Evaluation

Evaluation uses the greedy policy (argmax of each Q-table row) with no exploration, on fixed test windows:

1. The 409,856-step test sequence is divided into 2,049 possible non-overlapping blocks of 200 steps.
2. 100 of them are selected, spread evenly in chronological order from the start to the end of the test period (np.linspace).
3. Both algorithms are run on exactly the same 100 windows, and the code asserts that window lengths match.
4. The total reward of each window is recorded, together with action counts overall and per state.

Non-overlapping windows avoid counting the same test records more than once. Their total reward is the main performance measure; its mean and standard deviation across windows are reported.

# Chapter 6 Experimental Setup

## 6.1 Training Configuration

Table 6.1: Experimental Configuration

| Parameter | Value |
|---|---|
| Dataset | household_power_consumption.txt |
| Records (raw / cleaned) | 2,075,259 / 2,049,280 |
| Split | Chronological 80% / 20% |
| Training / test records | 1,639,424 / 409,856 |
| State variable | Global_active_power (kW) |
| Number of states | 5 (training-quantile bins) |
| State thresholds (kW) | 0.272, 0.400, 1.266, 1.742 |
| Number of actions | 3 |
| Algorithms | Q-Learning (off-policy), SARSA (on-policy) |
| Episodes | 500 |
| Maximum steps per episode | 200 |
| Learning rate (α) | 0.10 |
| Discount factor (γ) | 0.95 |
| ε start / minimum / decay | 1.0 / 0.01 / 0.995 per episode |
| Q-table initialisation | zeros (5 x 3) |
| Tie-breaking | random among equal best actions |
| Primary seed | 42 |
| Robustness seeds | 0, 1, 2, 3, 4 |
| Evaluation windows | 100 non-overlapping, 200 steps each |
| Evaluation policy | Greedy, deterministic |

## 6.2 Training Procedure

For each episode the agent resets the environment to a random start, then repeatedly chooses an action, receives the next state and reward and updates the Q-table until the episode ends. The cumulative reward and number of steps are stored, and ε is decayed at the end of the episode. Q-Learning and SARSA share the same environment, hyperparameters and random seeds, so differences in behaviour can only come from the update rule and the random trajectory it induces.

## 6.3 Fairness and Reproducibility Controls

- State thresholds are fitted on training data only.
- The split is chronological, so the test period is strictly later than the training period.

- Both algorithms use identical hyperparameters and the same seeds.
- Evaluation is greedy and uses identical, fixed windows for both algorithms.
- All reported numbers are produced by the executed notebook; none are typed in by hand.
- All tables, figures and CSV files are exported to the folder rl_case_study_outputs.

## 6.4 Software and Hardware Configuration

The experiment was run in Google Colab using Python with NumPy, pandas, matplotlib and seaborn. The learning algorithms are implemented from scratch with NumPy and need no deep-learning framework or GPU: one training run takes under four seconds. Timings depend on the Colab machine and its load and are indicative only.

# Chapter 7 Evaluation Metrics

Because RL has no single accuracy-style score, several complementary measures are used.

## 7.1 Episode Reward

The total reward of an episode (or evaluation window) of T steps is

$$R_{\text{episode}} = \sum_{t=1}^{T} r_t \tag{7.1}$$

With 200 steps per window, a mean reward of 288.92 corresponds to an average of about 1.44 reward units per step.

## 7.2 Mean and Standard Deviation of Test Reward

For the 100 evaluation windows the mean reward measures average policy quality and the sample standard deviation (ddof = 1) measures how much it varies from one period to another.

## 7.3 Moving-Average Training Reward

Training reward is smoothed with a 25-episode moving average to show the learning trend, and the mean of the last 50 training episodes summarises the end of training. These training rewards include exploratory actions and are therefore not directly comparable to greedy test rewards.

## 7.4 Policy Agreement

The greedy policy of each algorithm is the action with the largest Q-value in each state. Agreement is tested by comparing the two policies state by state.

## 7.5 Action Distribution

The number of times each action is chosen during evaluation shows the behaviour of the policy (for example, whether it ever allows higher consumption).

## 7.6 Robustness Across Seeds

The mean test reward of each of the five seeds is summarised by its mean and sample standard deviation (the SD of the seed-level means). A difference between algorithms is considered meaningful only if it is large compared with this variability.

## 7.7 Computational Cost

Training time is the wall-clock time of the training function for 500 episodes.

# Chapter 8 Results and Analysis

## 8.1 Learned Q-Tables

The Q-tables learned in the primary run (seed 42) are shown below; the highest value in each row (the greedy action) is the policy choice.

Table 8.1: Q-Learning Q-Table (seed 42)

| State | Reduce | Maintain | Allow Higher |
|---|---|---|---|
| Very Low | 25.904 | **28.719** | 25.627 |
| Low | 25.349 | **27.147** | 26.995 |
| Medium | **34.458** | 34.328 | 31.018 |
| High | **37.506** | 33.565 | 31.834 |
| Very High | 32.004 | **33.141** | 32.478 |

Table 8.2: SARSA Q-Table (seed 42)

| State | Reduce | Maintain | Allow Higher |
|---|---|---|---|
| Very Low | 23.544 | **25.566** | 23.339 |
| Low | 22.704 | **24.624** | 24.367 |
| Medium | **31.647** | 30.344 | 27.279 |
| High | **33.050** | 29.506 | 27.805 |
| Very High | 27.769 | **28.876** | 27.887 |

The two tables have the same structure: values are highest for the Medium and High states, and the best action in each state is the same for both algorithms. SARSA's values are systematically lower by about 2 to 5 units. This is consistent with the on-policy target: SARSA bootstraps from the value of the next action actually chosen, which is sometimes an exploratory and therefore worse action, whereas Q-Learning always bootstraps from the best next action. Because the Q-Learning values assume greedy behaviour afterwards, they are higher.

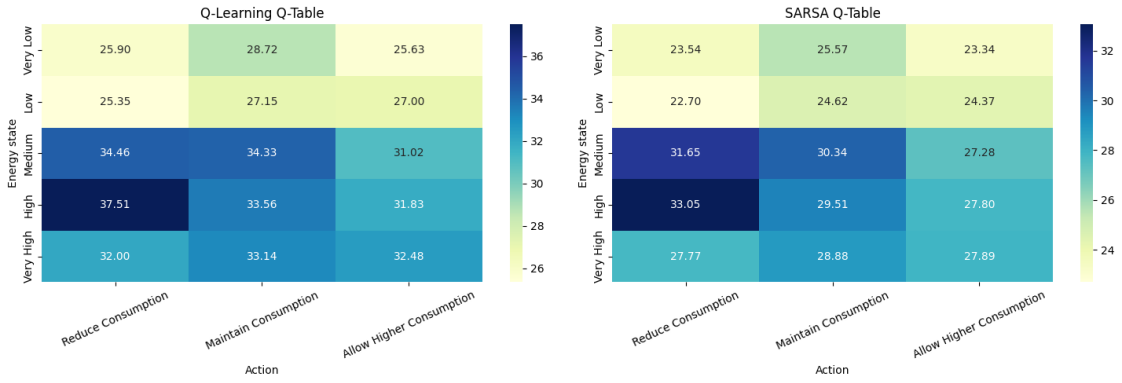



Figure 8.1: Q-Table Heatmaps of Q-Learning and SARSA (seed 42)

## 8.2 Learned Policies

Table 8.3: Greedy Policies Learned in the Primary Run

| Energy State | Q-Learning Action | SARSA Action | Same? |
|---|---|---|---|
| Very Low | Maintain | Maintain | Yes |
| Low | Maintain | Maintain | Yes |
| Medium | Reduce | Reduce | Yes |
| High | Reduce | Reduce | Yes |
| Very High | Maintain | Maintain | Yes |

In the primary run both algorithms learn exactly the same policy. The policy keeps consumption unchanged when demand is Very Low or Low, asks for a reduction when demand is Medium or High, and keeps consumption unchanged in the Very High state. It never allows higher consumption. The choice for the Very High state is examined critically in Section 10.3.

## 8.3 Training Behaviour

The 25-episode moving average of training reward starts at about −75 in the first episode, when ε is 1.0 and actions are essentially random, rises quickly to about 100 within the first 50 episodes, passes 200 by about episode 150 and reaches about 320 by episode 500. The two curves lie almost on top of each other throughout. The mean reward over the last 50 training episodes is 318.52 for Q-Learning and 319.89 for SARSA. The curve is still slowly rising at the end of training, which indicates that learning has not fully converged after 500 episodes.

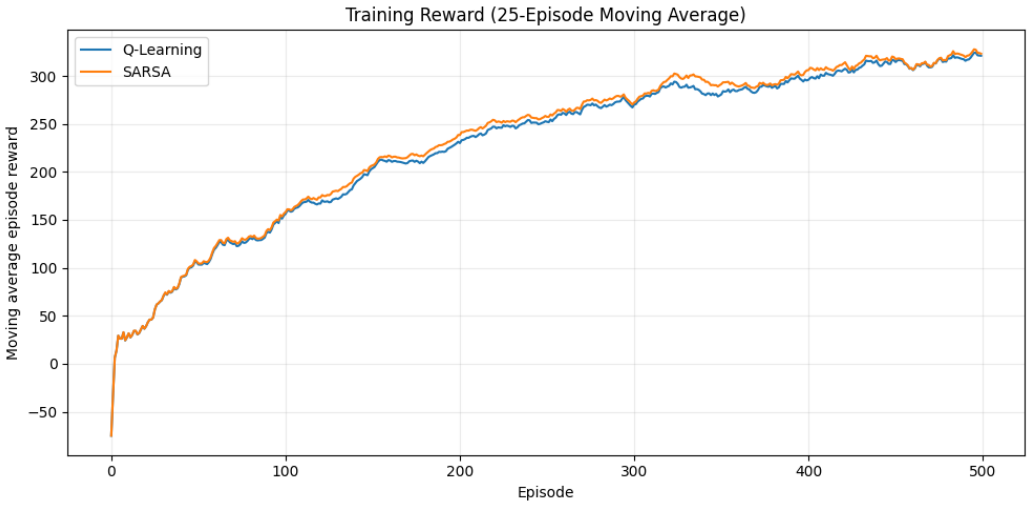



Figure 8.2: Training Reward (25-Episode Moving Average)

## 8.4 Test Performance in the Primary Run

Table 8.4: Primary-Run Results (seed 42, 100 test windows)

| Metric | Q-Learning | SARSA |
|---|---|---|
| Mean last-50 training reward | 318.52 | 319.89 |
| Mean test reward per window | 288.92 | 288.92 |
| SD of test reward per window | 90.10 | 90.10 |
| Training time (s) | 3.897 | 3.487 |

The test results are identical to the precision reported, which follows directly from the identical policies: the greedy policy is deterministic and both algorithms are run on exactly the same windows. The test reward per window (288.92) is lower than the late-training reward (about 319), because the two are measured on different data: the training reward is collected on the training period with roughly 8–10% random actions late in training, whereas the test reward is a purely greedy policy on a later period with a different state mix.

Table 8.5: Action Counts During Test Evaluation (100 windows x 200 steps)

| Action | Q-Learning | SARSA |
|---|---|---|
| Reduce Consumption | 10,278 | 10,278 |
| Maintain Consumption | 9,722 | 9,722 |
| Allow Higher Consumption | 0 | 0 |
| **Total decisions** | **20,000** | **20,000** |

The policy asks for a reduction in about 51% of the steps and maintains consumption in about 49%, which is roughly in line with the share of test records in the Medium and High states (about 48%).

## 8.5 Five-Seed Robustness Evaluation

Table 8.6: Results for Five Independent Seeds

| Seed | Q train reward (last 50) | SARSA train reward (last 50) | Q mean test reward | SARSA mean test reward | Q test SD | SARSA test SD | Same policy? |
|---|---|---|---|---|---|---|---|
| 0 | 326.03 | 315.06 | 369.965 | 231.615 | 29.115 | 88.676 | No |
| 1 | 314.77 | 314.77 | 325.405 | 369.965 | 88.990 | 29.115 | No |
| 2 | 318.05 | 319.44 | 218.665 | 278.350 | 100.449 | 76.900 | No |
| 3 | 319.24 | 323.72 | 312.660 | 369.965 | 77.636 | 29.115 | No |
| 4 | 311.88 | 315.07 | 369.965 | 369.965 | 29.115 | 29.115 | Yes |

Table 8.7: Summary Over Five Seeds

| Metric | Mean | Std |
|---|---|---|
| Q-Learning last-50 training reward | 317.994 | 5.336 |
| SARSA last-50 training reward | 317.612 | 3.927 |
| Q-Learning mean test reward | 319.332 | 61.933 |
| SARSA mean test reward | 323.972 | 65.110 |

Three observations stand out. First, training reward is almost identical for the two algorithms and stable across seeds (standard deviations of 5.3 and 3.9). Second, test reward is very unstable across seeds: individual seed results range from 218.7 to 370.0, a spread much larger than the 4.64-unit difference between the algorithm means. Third, the same value, 369.965 (with a window SD of 29.115), appears in five of the ten runs (seeds 0 and 4 for Q-Learning; seeds 1, 3 and 4 for SARSA), which suggests that it is the reward of one particular policy that some runs find and others miss.

Only one of the five seeds (seed 4) produced identical greedy policies for the two algorithms, so the identical policies of the primary run are not typical; with a different seed the algorithms often end in different policies. These differences are, however, not systematic: Q-Learning is better in seed 0, SARSA in seeds 1, 2 and 3.

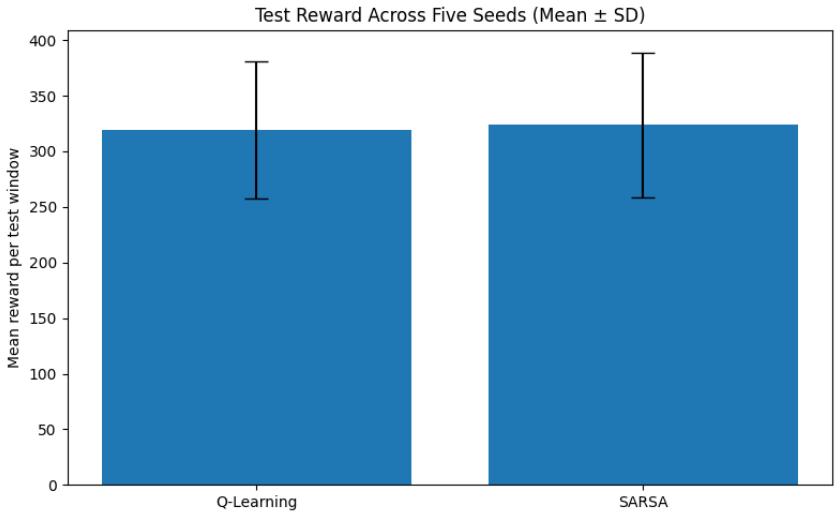



Figure 8.3: Test Reward Across Five Seeds (Mean ± SD)

## 8.6 Final Comparison

Table 8.8: Final Comparison of Q-Learning and SARSA

| Measure | Q-Learning | SARSA |
|---|---|---|
| Mean last-50 training reward (primary run) | 318.52 | 319.89 |
| Mean test reward (primary run) | 288.92 | 288.92 |
| SD of test reward (primary run) | 90.10 | 90.10 |
| Training time, primary run (s) | 3.897 | 3.487 |
| Mean test reward across five seeds | 319.332 | 323.972 |
| SD of seed-level test means | 61.933 | 65.110 |

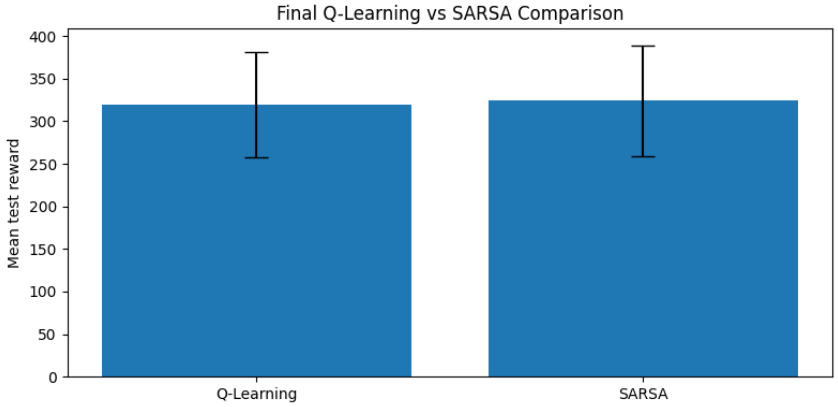



Figure 8.4: Final Q-Learning vs SARSA Comparison (Mean ± SD over Five Seeds)

SARSA's mean test reward across seeds is 4.640 units higher than that of Q-Learning. With seed-level standard deviations above 60 and only five seeds, this difference is about 7% of one standard deviation and cannot be regarded as evidence that either algorithm is better. SARSA was also about 0.41 s (roughly 10%) faster in the primary run, but with a single timing on a shared Colab machine this is not a reliable difference either.

# Chapter 9 Sample Policy Outputs

## 9.1 Recommended Action per Consumption State

The learned policy can be read as a small rule book. Table 9.1 lists the action recommended in each state by the policy of the primary run (identical for both algorithms), together with the greedy Q-values.

Table 9.1: Policy Learned in the Primary Run

| Consumption state | Power range (kW) | Recommended action | Q-Learning value | SARSA value |
|---|---|---|---|---|
| Very Low | below 0.272 | Maintain | 28.719 | 25.566 |
| Low | 0.272 – 0.400 | Maintain | 27.147 | 24.624 |
| Medium | 0.400 – 1.266 | Reduce | 34.458 | 31.647 |
| High | 1.266 – 1.742 | Reduce | 37.506 | 33.050 |
| Very High | 1.742 and above | Maintain | 33.141 | 28.876 |

## 9.2 Interpretation of the Sample Outputs

The Very Low, Low, Medium and High recommendations follow the reward rules sensibly. When demand is Very Low or Low, Maintain gives a base reward of +1 while Reduce gives no base reward and Allow Higher gives none either, so keeping consumption unchanged is the natural choice. When demand is High, Reduce gives a base reward of +2 and a further +1 if consumption falls, whereas Maintain scores −1 and Allow Higher −2, so Reduce is clearly best. In the Medium state, Reduce and Maintain have almost equal values (34.458 against 34.328 for Q-Learning), so the preference is weak and is the state in which the learned choice is most likely to change with a different seed.

The Very High recommendation (Maintain) is the surprising one and is analysed in Section 10.3.

# Chapter 10 Discussion

## 10.1 Comparison of Q-Learning and SARSA

The results show that Q-Learning and SARSA behave almost identically on this task. Their training curves overlap, their last-50-episode training rewards differ by only 1.37 units (318.52 and 319.89), their primary-run policies and test rewards are identical, and over five seeds their mean test rewards (319.33 and 323.97) differ by 4.64 units against a seed-to-seed standard deviation of more than 60. Neither algorithm can be declared better. The only consistent difference is in the magnitude of the Q-values, which are lower for SARSA because its targets include exploratory next actions.

## 10.2 Why the Two Algorithms Behave So Similarly

The theoretical distinction between on-policy and off-policy learning matters when exploratory actions can lead to costly outcomes, as in the cliff-walking example, where SARSA learns a safer path than Q-Learning. In the present replay environment an exploratory action cannot lead to a worse future, because the next state is the next recorded consumption category whatever the action is. Consequently

$$Q^*(s,a) = \mathbb{E}[r(s,a,s')] + \gamma \mathbb{E}[V^*(s')] \tag{10.1}$$

where the second term is the same for every action, so the ranking of actions in each state is determined only by the expected immediate reward. The problem is effectively a contextual-bandit problem, and neither bootstrapping nor the choice between a maximum and a sampled next action changes which action is best. The two algorithms differ only in the noise their targets carry, which explains why their outcomes are interchangeable up to random variation.

This also means that the discount factor $\gamma$ does not change the optimal policy here; it only scales the Q-values (a long-run average reward of about 1.5 per step gives values of about $1.5 / (1 - 0.95) = 30$, consistent with the table values of 25–37).

## 10.3 Sanity Check of the Learned Policy

Because the action does not affect the transitions, the best action in each state can be checked directly against the reward rules. In the Very High state (4) the next state can never be higher, so Reduce always earns at least +2, Maintain earns at most −0.5 and Allow Higher earns exactly −2. Reduce therefore dominates in that state. The Maintain recommendation of the primary run (Q-values 33.141, 32.478 and 32.004 for Maintain, Allow and Reduce) is thus inconsistent with the reward rules and is most likely a consequence of incomplete convergence: after 500 episodes $\varepsilon$ is still about 0.08, the training curve is still rising, and the bootstrapped values carry noise that is comparable in size to the differences between the actions.

The test results support this reading. The test reward value 369.965 shared by several seed runs exceeds the primary-run value of 288.92 by 81.05 units per window, i.e. 0.405 per step. Very High states make up 15.9% of the test steps (65,009 of 409,856), so the gain per Very High step would be about 2.56 units, which matches the minimum advantage of Reduce over Maintain in that state (at least 2.5). This is consistent with the 369.965 runs having learned to reduce in the Very High state, although the individual policies of the other seeds were not printed in the notebook and this should be checked directly.

This finding does not change the main conclusion about the two algorithms, since both made the same choice in the primary run, but it shows that the learning was not fully converged and that the primary-run test reward is not the best attainable under the reward design.

## 10.4 Interpretation of the Seed Variability

The wide spread of test rewards across seeds (218.7 to 370.0) comes from small differences in the final greedy policy, particularly in states where the Q-values of two actions are close (Medium, and the Very High state discussed above). Because the policy is deterministic and the test windows are fixed, a single flipped action in one state changes the average test reward by a large and systematic amount. Training reward, which averages over exploratory behaviour, is far more stable (standard deviation about 4–5). This shows that the five-seed analysis is necessary: a single run would have given a misleading picture, and the identical results of the primary run were partly a coincidence.

## 10.5 Computational Considerations

Both algorithms need only a 5 x 3 table and about 100,000 updates, and each finishes in under four seconds. Memory and compute are negligible, so the choice between them should be made on behaviour and safety considerations, not cost. The 10% speed difference measured in the primary run (3.487 s against 3.897 s) is within what normal system noise can produce.

## 10.6 Final Interpretation

The study supports three conclusions. First, with the stated reward and environment, Q-Learning and SARSA are practically equivalent. Second, 500 episodes with ε decay of 0.995 do not guarantee full convergence, and the robustness analysis is important for seeing this. Third, the results describe the behaviour of the algorithms on a replay task with hand-made rewards and cannot be used to claim real electricity savings.

# Chapter 11 Limitations

The study has the following limitations:

1. **Action-independent transitions.** The environment replays recorded data, so the agent's actions do not change future consumption. The task reduces to choosing the action with the highest expected immediate reward, and the strengths of RL in handling delayed consequences are not exercised.
2. **Manually designed reward.** The reward has arbitrary units and encodes the designer's assumptions. It is not based on tariffs, peak demand, comfort or carbon intensity, and reward totals are not kWh or currency.
3. **No evidence of real savings.** Because of limitations 1 and 2, the experiment does not establish actual electricity savings or deployment performance.
4. **Coarse state representation.** A single variable (global active power) is discretised into five states. Time of day, day of week, season, appliance sub-metering and voltage are ignored, so the agent cannot learn time-dependent behaviour.
5. **Limited training.** Five hundred episodes of 200 steps use only about 100,000 of the 1.64 million training records, and the training curve is still rising at the end. The suboptimal Very High action of the primary run illustrates incomplete convergence.
6. **Limited statistical evidence.** Five seeds give only an initial robustness check. The differences between the algorithms are much smaller than the seed variability, and no significance test was applied.
7. **Fixed hyperparameters.** The learning rate, discount factor and ε schedule were fixed in advance and not tuned or compared across settings.
8. **Single household and period.** The data come from one household in one location between 2006 and 2010. Patterns may not transfer to other homes or later years.
9. **Distribution shift between train and test.** The test period has a different state mix (Medium 25.9%, Very Low 15.6%) from the training period (about 20% each), so test rewards are not directly comparable with training rewards.
10. **Timing environment.** Training times were measured once on a shared Google Colab runtime and are indicative only.

# Chapter 12 Future Enhancements

Possible directions for extending this work are:

1. **Action-dependent simulator.** Build and validate a model in which the chosen action changes future consumption (for example, load-shifting appliances), so that on-policy and off-policy differences can become visible.
2. **Realistic reward design.** Define rewards from time-of-use tariffs, peak-demand charges, comfort constraints or carbon intensity.
3. **Richer states.** Include hour of day, day of week, season, recent consumption trend and sub-metering readings, possibly with finer discretisation.
4. **Convergence study.** Train for more episodes with slower $\varepsilon$ decay and a decaying learning rate, and track Q-table changes to show convergence.
5. **Exact reference solution.** Because transitions here do not depend on actions, the optimal policy can be computed directly from empirical transition counts and compared with the learned policies as ground truth.
6. **Hyperparameter study.** Compare several values of $\alpha$, $\gamma$ and the $\varepsilon$ schedule, and report sensitivity.
7. **More seeds and statistical testing.** Use 30 or more seeds with confidence intervals or non-parametric tests.
8. **Further algorithms.** Compare Expected SARSA, Double Q-Learning, n-step methods and eligibility traces once the environment is validated.
9. **Function approximation.** Use tile coding or a Deep Q-Network to handle continuous or high-dimensional states.
10. **Multiple households and longer horizons.** Evaluate across several households and longer chronological windows.
11. **Deployment.** Build a dashboard that turns the current consumption level into a recommended action, with human oversight.

# Chapter 13 Conclusion

This case study compared tabular Q-Learning and SARSA for energy consumption management using 2,049,280 cleaned minute-level household power records. Global active power was discretised into five training-derived quantile states, three actions were defined, and both algorithms were trained under identical settings (500 episodes, 200 steps, $\alpha = 0.10$, $\gamma = 0.95$, $\varepsilon$-greedy with decay 0.995) and evaluated on 100 identical, non-overlapping chronological test windows.

In the primary run both algorithms learned the same greedy policy and achieved a mean test reward of 288.92 per window (SD 90.10); training took 3.897 s for Q-Learning and 3.487 s for SARSA. Over five seeds, the mean test rewards were 319.33 (SD 61.93) for Q-Learning and 323.97 (SD 65.11) for SARSA. The difference of 4.64 units is small compared with the seed variability, so the study finds no meaningful performance difference between the two algorithms. SARSA's Q-values were consistently lower than Q-Learning's, as expected from its on-policy target.

The analysis also showed the limits of the experiment. Because actions do not change future consumption, the problem is effectively a contextual bandit and the usual contrast between on-policy and off-policy learning does not appear. A reward-rule check suggests that the Maintain action learned for the Very High state in the primary run is not optimal, which points to incomplete convergence and underlines the value of multi-seed evaluation. All rewards are arbitrary scores and do not represent real savings.

Future work should develop an action-dependent simulator with realistic rewards, richer state information, longer training and more seeds before drawing conclusions about real-world energy management.

# References

[1] Sutton, R. S., and Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

[2] Watkins, C. J. C. H., and Dayan, P. (1992). Q-Learning. Machine Learning, 8(3–4), 279–292.

[3] Watkins, C. J. C. H. (1989). Learning from Delayed Rewards. PhD thesis, University of Cambridge.

[4] Rummery, G. A., and Niranjan, M. (1994). On-Line Q-Learning Using Connectionist Systems. Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department.

[5] Singh, S., Jaakkola, T., Littman, M. L., and Szepesvári, C. (2000). Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms. Machine Learning, 38(3), 287–308.

[6] Mnih, V., et al. (2015). Human-Level Control through Deep Reinforcement Learning. Nature, 518, 529–533.

[7] Hebrail, G., and Berard, A. (2006). Individual Household Electric Power Consumption [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C58K54 (accessed October 2026).

[8] Harris, C. R., et al. (2020). Array Programming with NumPy. Nature, 585, 357–362.

[9] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

[10] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

[11] Waskom, M. L. (2021). seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 3021.

# Appendix A Source Code

The complete Python code used for the experiments is listed below. It was executed as a Google Colab notebook; the cells appear in execution order.

### Cell 1: Imports and configuration

```
import time
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

SEED = 42
N_EPISODES = 500
MAX_STEPS = 200
ALPHA = 0.10
GAMMA = 0.95
EPSILON_START = 1.0
EPSILON_MIN = 0.01
EPSILON_DECAY = 0.995
N_EVAL_WINDOWS = 100
SEEDS = [0, 1, 2, 3, 4]
OUTPUT_DIR = Path("rl_case_study_outputs")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
print("Configuration ready.")
print(f"Episodes={N_EPISODES}, max steps={MAX_STEPS}, alpha={ALPHA}, gamma={GAMMA}")
```

### Cell 2: Load the dataset

```
def get_data_path():
    candidate = Path("household_power_consumption.txt")
    if candidate.exists():
        return candidate
    try:
        from google.colab import files
        uploaded = files.upload()
        if not uploaded:
            raise FileNotFoundError("No dataset uploaded.")
        name = next(iter(uploaded))
        return Path(name)
    except ImportError as exc:
        raise FileNotFoundError(
            "Place household_power_consumption.txt beside this notebook, "
            "or upload it in Google Colab."
        ) from exc

data_path = get_data_path()
df = pd.read_csv(data_path, sep=";", na_values=["?", ""], low_memory=False)
print("Dataset file:", data_path)
print("Original shape:", df.shape)
print("Columns:", df.columns.tolist())
display(df.head())
```

### Cell 3: Validate, clean and sort records

```
numeric_cols = [
    "Global_active_power", "Global_reactive_power", "Voltage",
    "Global_intensity", "Sub_metering_1", "Sub_metering_2", "Sub_metering_3"
]
required_cols = ["Date", "Time", *numeric_cols]
missing_cols = [col for col in required_cols if col not in df.columns]
if missing_cols:
    raise ValueError(f"Dataset is missing required columns: {missing_cols}")

df["Datetime"] = pd.to_datetime(
    df["Date"].astype(str) + " " + df["Time"].astype(str),
    dayfirst=True, errors="coerce"
)
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

rows_before = len(df)
df = (df.dropna(subset=["Datetime", *numeric_cols])
```

```
        .sort_values("Datetime")
        .drop_duplicates(subset=["Datetime"])
        .reset_index(drop=True))
rows_removed = rows_before - len(df)

if len(df) < 1000:
    raise ValueError("Too few valid rows remain after cleaning.")

print(f"Rows before cleaning: {rows_before:,}")
print(f"Rows after cleaning:  {len(df):,}")
print(f"Rows removed:         {rows_removed:,}")
print("Missing values in required columns:",
      int(df[["Datetime", *numeric_cols]].isna().sum().sum()))
print("Duplicate timestamps:", int(df["Datetime"].duplicated().sum()))
display(df.head())
```

## Cell 4: Exploratory data analysis

```
display(df[numeric_cols].describe().T.round(3))

sample_step = max(1, len(df) // 20000)
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(df["Datetime"].iloc[::sample_step],
        df["Global_active_power"].iloc[::sample_step], linewidth=0.6)
ax.set(title="Household Global Active Power Over Time",
       xlabel="Date", ylabel="Global active power (kW)")
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "energy_time_series.png", dpi=160)
plt.show()

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(df["Global_active_power"], bins=50)
ax.set(title="Distribution of Global Active Power",
       xlabel="Global active power (kW)", ylabel="Frequency")
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "energy_distribution.png", dpi=160)
plt.show()
```

## Cell 5: Chronological split and state discretisation

```
split_idx = int(0.80 * len(df))
train_power = df["Global_active_power"].iloc[:split_idx].to_numpy(dtype=float)
test_power = df["Global_active_power"].iloc[split_idx:].to_numpy(dtype=float)
test_datetimes = df["Datetime"].iloc[split_idx:].reset_index(drop=True)

STATE_NAMES = ["Very Low", "Low", "Medium", "High", "Very High"]
ACTION_NAMES = ["Reduce Consumption", "Maintain Consumption", "Allow Higher Consumption"]
N_STATES, N_ACTIONS = len(STATE_NAMES), len(ACTION_NAMES)

state_edges = np.quantile(train_power, [0.2, 0.4, 0.6, 0.8])
train_states = np.digitize(train_power, state_edges).astype(np.int64)
test_states = np.digitize(test_power, state_edges).astype(np.int64)

print(f"Training records: {len(train_states):,} ({len(train_states)/len(df):.1%})")
print(f"Test records:     {len(test_states):,} ({len(test_states)/len(df):.1%})")
print("Training-derived thresholds (kW):", np.round(state_edges, 3))

state_distribution = pd.DataFrame({
    "State": STATE_NAMES,
    "Training count": np.bincount(train_states, minlength=N_STATES),
    "Test count": np.bincount(test_states, minlength=N_STATES)
})
display(state_distribution)

ax = state_distribution.set_index("State")[["Training count", "Test count"]].plot(
    kind="bar", figsize=(9, 4)
)
ax.set(title="Energy State Counts: Training vs Test", ylabel="Records")
plt.xticks(rotation=20)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "state_distribution.png", dpi=160)
plt.show()
```

## Cell 6: Replay environment and reward function

```
class EnergyReplayEnvironment:
    def __init__(self, states, max_steps=MAX_STEPS):
        self.states = np.asarray(states, dtype=np.int64)
        if self.states.ndim != 1 or len(self.states) < 2:
            raise ValueError("states must be a 1-D array with at least two elements")
```

```
        if np.any((self.states < 0) | (self.states >= N_STATES)):
            raise ValueError("State IDs must be in the range 0..4")
        if max_steps < 1:
            raise ValueError("max_steps must be positive")
        self.n_states = N_STATES
        self.n_actions = N_ACTIONS
        self.max_steps = min(int(max_steps), len(self.states) - 1)
        self.current_index = 0
        self.steps = 0

    def reset(self, rng, start_index=None):
        max_start = len(self.states) - self.max_steps - 1
        if max_start < 0:
            raise ValueError("State sequence is too short for an episode")
        if start_index is None:
            self.current_index = int(rng.integers(0, max_start + 1))
        else:
            self.current_index = int(start_index)
        if not 0 <= self.current_index <= max_start:
            raise ValueError(f"start_index must be between 0 and {max_start}")
        self.steps = 0
        return int(self.states[self.current_index])

    @staticmethod
    def reward_for(state, next_state, action):
        # Transparent rule-based scores; not physical energy measurements.
        if action == 0:  # Reduce consumption
            return float((2 if state >= 2 else 0) +
                         (1 if next_state < state else (-1 if next_state > state else 0)))
        if action == 1:  # Maintain consumption
            return float((1 if state <= 2 else -1) +
                         (0.5 if abs(next_state - state) <= 1 else 0))
        if action == 2:  # Allow higher consumption
            return float((-2 if state >= 2 else 0) +
                         (-1 if next_state > state else 0))
        raise ValueError("Action must be 0, 1, or 2")

    def step(self, action):
        action = int(action)
        if not 0 <= action < self.n_actions:
            raise ValueError("Invalid action")
        state = int(self.states[self.current_index])
        self.current_index += 1
        next_state = int(self.states[self.current_index])
        reward = self.reward_for(state, next_state, action)
        self.steps += 1
        done = (self.steps >= self.max_steps or
                self.current_index >= len(self.states) - 1)
        return next_state, reward, done

train_env = EnergyReplayEnvironment(train_states)
print(f"Environment ready: {N_STATES} states, {N_ACTIONS} actions, "
      f"{train_env.max_steps} steps/episode")
```

## Cell 7: Action selection and helper functions

```
def choose_action(q_table, state, epsilon, rng):
    if rng.random() < epsilon:
        return int(rng.integers(q_table.shape[1]))
    # Random tie-breaking during training reduces first-action bias at equal values.
    best_actions = np.flatnonzero(np.isclose(q_table[state], np.max(q_table[state])))
    return int(rng.choice(best_actions))

def moving_average(values, window=25):
    return pd.Series(values).rolling(window=window, min_periods=1).mean().to_numpy()
```

## Cell 8: Q-Learning

```
def train_q_learning(env, episodes=N_EPISODES, alpha=ALPHA, gamma=GAMMA,
                     epsilon_start=EPSILON_START, epsilon_min=EPSILON_MIN,
                     epsilon_decay=EPSILON_DECAY, seed=SEED):
    rng = np.random.default_rng(seed)  # Controls starts and actions
    q = np.zeros((env.n_states, env.n_actions), dtype=float)
    episode_rewards, episode_lengths = [], []
    epsilon = epsilon_start

    for _ in range(episodes):
        state = env.reset(rng)
        total_reward, steps = 0.0, 0
        while True:
```

```
            action = choose_action(q, state, epsilon, rng)
            next_state, reward, done = env.step(action)
            # Off-policy target: maximum next-state action value.
            target = reward if done else reward + gamma * np.max(q[next_state])
            q[state, action] += alpha * (target - q[state, action])
            state = next_state
            total_reward += reward
            steps += 1
            if done:
                break
        episode_rewards.append(total_reward)
        episode_lengths.append(steps)
        epsilon = max(epsilon_min, epsilon * epsilon_decay)

    return q, np.asarray(episode_rewards), np.asarray(episode_lengths)

start = time.perf_counter()
q_table, q_rewards, q_lengths = train_q_learning(train_env, seed=SEED)
q_training_time = time.perf_counter() - start
print(f"Q-Learning training time: {q_training_time:.3f} seconds")
display(pd.DataFrame(q_table, index=STATE_NAMES, columns=ACTION_NAMES).round(3))
```

## Cell 9: SARSA

```
def train_sarsa(env, episodes=N_EPISODES, alpha=ALPHA, gamma=GAMMA,
                epsilon_start=EPSILON_START, epsilon_min=EPSILON_MIN,
                epsilon_decay=EPSILON_DECAY, seed=SEED):
    rng = np.random.default_rng(seed)  # Controls starts and actions
    q = np.zeros((env.n_states, env.n_actions), dtype=float)
    episode_rewards, episode_lengths = [], []
    epsilon = epsilon_start

    for _ in range(episodes):
        state = env.reset(rng)
        action = choose_action(q, state, epsilon, rng)
        total_reward, steps = 0.0, 0
        while True:
            next_state, reward, done = env.step(action)
            if done:
                target, next_action = reward, None
            else:
                next_action = choose_action(q, next_state, epsilon, rng)
                # On-policy target: value of the next action actually selected.
                target = reward + gamma * q[next_state, next_action]
            q[state, action] += alpha * (target - q[state, action])
            total_reward += reward
            steps += 1
            if done:
                break
            state, action = next_state, next_action

        episode_rewards.append(total_reward)
        episode_lengths.append(steps)
        epsilon = max(epsilon_min, epsilon * epsilon_decay)

    return q, np.asarray(episode_rewards), np.asarray(episode_lengths)

start = time.perf_counter()
sarsa_table, sarsa_rewards, sarsa_lengths = train_sarsa(train_env, seed=SEED)
sarsa_training_time = time.perf_counter() - start
print(f"SARSA training time: {sarsa_training_time:.3f} seconds")
display(pd.DataFrame(sarsa_table, index=STATE_NAMES, columns=ACTION_NAMES).round(3))
```

## Cell 10: Training reward curves

```
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(moving_average(q_rewards), label="Q-Learning")
ax.plot(moving_average(sarsa_rewards), label="SARSA")
ax.set(title="Training Reward (25-Episode Moving Average)",
       xlabel="Episode", ylabel="Moving average episode reward")
ax.legend()
ax.grid(alpha=0.25)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "training_reward_comparison.png", dpi=160)
plt.show()

print(f"Q-Learning last-50 training reward: {q_rewards[-50:].mean():.3f}")
print(f"SARSA last-50 training reward:      {sarsa_rewards[-50:].mean():.3f}")
```

## Cell 11: Held-out chronological evaluation

```
def make_test_windows(states, n_windows=N_EVAL_WINDOWS, window_size=MAX_STEPS):
    states = np.asarray(states)
    if len(states) < window_size + 1:
        raise ValueError("Test period is too short for the requested evaluation window.")
    # Partition the test timeline into chronological, non-overlapping windows.
    max_windows = (len(states) - 1) // window_size
    count = min(int(n_windows), max_windows)
    if count < 1:
        raise ValueError("No complete evaluation windows available.")
    starts = np.linspace(0, (max_windows - count) * window_size,
                         num=count, dtype=int)
    starts = np.unique(starts)
    return [(int(s), int(min(window_size, len(states) - s - 1))) for s in starts]

def evaluate_policy(states, q_table, windows):
    rewards, lengths = [], []
    action_counts = np.zeros(N_ACTIONS, dtype=int)
    state_action_counts = np.zeros((N_STATES, N_ACTIONS), dtype=int)

    for start_index, length in windows:
        env = EnergyReplayEnvironment(states, max_steps=length)
        # A dedicated generator is passed only for API consistency; start is fixed.
        eval_rng = np.random.default_rng(0)
        state = env.reset(eval_rng, start_index=start_index)
        total_reward, steps = 0.0, 0
        while True:
            action = int(np.argmax(q_table[state]))  # deterministic greedy evaluation
            action_counts[action] += 1
            state_action_counts[state, action] += 1
            next_state, reward, done = env.step(action)
            total_reward += reward
            steps += 1
            state = next_state
            if done:
                break
        rewards.append(total_reward)
        lengths.append(steps)

    return np.asarray(rewards), np.asarray(lengths), action_counts, state_action_counts

test_windows = make_test_windows(test_states)
q_eval, q_eval_lengths, q_action_counts, q_state_action_counts = evaluate_policy(
    test_states, q_table, test_windows)
sarsa_eval, sarsa_eval_lengths, sarsa_action_counts, sarsa_state_action_counts = evaluate_policy(
    test_states, sarsa_table, test_windows)

assert np.array_equal(q_eval_lengths, sarsa_eval_lengths), \
    "Evaluation windows differ between algorithms"
print(f"Shared test windows: {len(test_windows)}")
print(f"Steps per window: {q_eval_lengths.min()}-{q_eval_lengths.max()}")
single_run_results = pd.DataFrame({
    "Algorithm": ["Q-Learning", "SARSA"],
    "Mean last-50 training reward": [q_rewards[-50:].mean(), sarsa_rewards[-50:].mean()],
    "Mean test reward per window": [q_eval.mean(), sarsa_eval.mean()],
    "SD test reward per window": [q_eval.std(ddof=1), sarsa_eval.std(ddof=1)],
    "Training time (seconds)": [q_training_time, sarsa_training_time]
})
display(single_run_results.round(3))
display(pd.DataFrame({"Q-Learning": q_action_counts, "SARSA": sarsa_action_counts},
                     index=ACTION_NAMES))
print("Same greedy policy:", np.array_equal(q_table.argmax(axis=1), sarsa_table.argmax(axis=1)))
print("Same evaluation rewards:", np.allclose(q_eval, sarsa_eval))
```

## Cell 12: Learned policies and Q-table heatmaps

```
policy_table = pd.DataFrame({
    "Energy state": STATE_NAMES,
    "Q-Learning action": [ACTION_NAMES[int(np.argmax(q_table[s]))] for s in range(N_STATES)],
    "SARSA action": [ACTION_NAMES[int(np.argmax(sarsa_table[s]))] for s in range(N_STATES)]
})
policy_table["Same action"] = policy_table["Q-Learning action"] == policy_table["SARSA action"]
display(policy_table)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
for ax, table, title in [
    (axes[0], q_table, "Q-Learning Q-Table"),
    (axes[1], sarsa_table, "SARSA Q-Table")
]:
    sns.heatmap(table, annot=True, fmt=".2f", cmap="YlGnBu",
                xticklabels=ACTION_NAMES, yticklabels=STATE_NAMES, ax=ax)
    ax.set(title=title, xlabel="Action", ylabel="Energy state")
```

```
    ax.tick_params(axis="x", rotation=25)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "q_table_heatmaps.png", dpi=160)
plt.show()
```

## Cell 13: Five-seed robustness evaluation

```
robustness_rows = []
for seed in SEEDS:
    q_seed, qr_seed, _ = train_q_learning(train_env, seed=seed)
    s_seed, sr_seed, _ = train_sarsa(train_env, seed=seed)
    q_test_seed, _, _, _ = evaluate_policy(test_states, q_seed, test_windows)
    s_test_seed, _, _, _ = evaluate_policy(test_states, s_seed, test_windows)

    robustness_rows.append({
        "Seed": seed,
        "Q train reward (last 50)": qr_seed[-50:].mean(),
        "SARSA train reward (last 50)": sr_seed[-50:].mean(),
        "Q mean test reward": q_test_seed.mean(),
        "SARSA mean test reward": s_test_seed.mean(),
        "Q test reward SD": q_test_seed.std(ddof=1),
        "SARSA test reward SD": s_test_seed.std(ddof=1),
        "Same greedy policy": np.array_equal(q_seed.argmax(axis=1), s_seed.argmax(axis=1))
    })

robustness = pd.DataFrame(robustness_rows)
display(robustness.round(3))

summary_metrics = [
    "Q train reward (last 50)", "SARSA train reward (last 50)",
    "Q mean test reward", "SARSA mean test reward"
]
robustness_summary = robustness[summary_metrics].agg(["mean", "std"]).T
display(robustness_summary.round(3))

fig, ax = plt.subplots(figsize=(8, 5))
means = [robustness["Q mean test reward"].mean(), robustness["SARSA mean test reward"].mean()]
stds = [robustness["Q mean test reward"].std(ddof=1),
        robustness["SARSA mean test reward"].std(ddof=1)]
ax.bar(["Q-Learning", "SARSA"], means, yerr=stds, capsize=8)
ax.set(title="Test Reward Across Five Seeds (Mean ± SD)",
       ylabel="Mean reward per test window")
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "multi_seed_test_comparison.png", dpi=160)
plt.show()
```

## Cell 14: Final comparison

```
final_comparison = pd.DataFrame({
    "Algorithm": ["Q-Learning", "SARSA"],
    "Mean last-50 training reward (single run)": [q_rewards[-50:].mean(), sarsa_rewards[-
50:].mean()],
    "Mean test reward (single run)": [q_eval.mean(), sarsa_eval.mean()],
    "SD test reward (single run)": [q_eval.std(ddof=1), sarsa_eval.std(ddof=1)],
    "Training time (seconds, single run)": [q_training_time, sarsa_training_time],
    "Mean test reward across seeds": [
        robustness["Q mean test reward"].mean(),
        robustness["SARSA mean test reward"].mean()
    ],
    "SD of seed-level test means": [
        robustness["Q mean test reward"].std(ddof=1),
        robustness["SARSA mean test reward"].std(ddof=1)
    ]
})
display(final_comparison.round(3))

fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(final_comparison["Algorithm"], final_comparison["Mean test reward across seeds"],
       yerr=final_comparison["SD of seed-level test means"], capsize=8)
ax.set(title="Final Q-Learning vs SARSA Comparison", ylabel="Mean test reward")
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "final_comparison.png", dpi=160)
plt.show()

q_mean = final_comparison.loc[0, "Mean test reward across seeds"]
sarsa_mean = final_comparison.loc[1, "Mean test reward across seeds"]
difference = sarsa_mean - q_mean
print(f"SARSA minus Q-Learning mean test reward: {difference:.3f}")
print("Interpretation: compare the mean and variability together; "
```

```
      "five seeds provide limited evidence.")
print("Reward is an arbitrary score under the defined rules, not measured kWh or cost saved.")
```

## Cell 15: Export results

```
final_comparison.to_csv(OUTPUT_DIR / "final_comparison.csv", index=False)
single_run_results.to_csv(OUTPUT_DIR / "single_run_results.csv", index=False)
robustness.to_csv(OUTPUT_DIR / "multi_seed_results.csv", index=False)
policy_table.to_csv(OUTPUT_DIR / "learned_policies.csv", index=False)
state_distribution.to_csv(OUTPUT_DIR / "state_distribution.csv", index=False)
pd.DataFrame(q_table, index=STATE_NAMES, columns=ACTION_NAMES).to_csv(
    OUTPUT_DIR / "q_learning_table.csv")
pd.DataFrame(sarsa_table, index=STATE_NAMES, columns=ACTION_NAMES).to_csv(
    OUTPUT_DIR / "sarsa_table.csv")
pd.DataFrame({
    "Episode": np.arange(1, N_EPISODES + 1),
    "Q-Learning reward": q_rewards,
    "SARSA reward": sarsa_rewards
}).to_csv(OUTPUT_DIR / "training_rewards.csv", index=False)

print("Exported files to:", OUTPUT_DIR.resolve())
for path in sorted(OUTPUT_DIR.iterdir()):
    print("-", path.name)
```